TRƯỜNG ĐẠI HỌC BÁCH KHOA – ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Môn học: **Lập trình Đa nền tảng**

«Chủ đề 03: Responsive và Adaptive UI»

Tối ưu hóa Giao diện Đa nền tảng: Thích ứng Kích thước & Phương thức Nhập liệu

Sinh viên thực hiện:
1. Lê Anh Khoa
2. Ngô Quang Thế

Đà Nẵng, 2026

Mục lục Nội dung Báo cáo

PHẦN I: NỀN TẢNG, CƠ CHẾ & QUY TRÌNH

Thuyết trình: Lê Anh Khoa

- **Responsive vs. Adaptive:** Phân định chuẩn theo 2 nguồn
- **Bảng đối chiếu chuyên sâu** theo 4 tiêu chí cốt lõi
- **Cơ chế MediaQuery:** `sizeOf` (Flutter 3.10+) & Đơn vị dp
- **Cơ chế LayoutBuilder:** Quy tắc vàng *BoxConstraints*
- **Quy trình 3 bước chuẩn:** Abstract → Measure → Branch
- **So sánh phạm vi** & Breakpoints Material Design 3

PHẦN II: THIẾT KẾ, KIẾN TRÚC & DEMO

Thuyết trình: Ngô Quang Thế

- **Canonical Layouts chuẩn M3:** Feed, List-Detail, Supporting Pane
- **Phương thức nhập liệu:** Touch (≥ 48dp) vs. Pointer/Keyboard
- **Kiến trúc ứng dụng** & Sơ đồ luồng quyết định 3 tầng
- **Phân tích Mã nguồn Trọng tâm**
- **Kịch bản Thực nghiệm** & Bảng Live Demo đo đạc thực tế
- **Tổng kết Đề tài** & Hạn chế Nghiên cứu
- **Tài liệu tham khảo (References)**

Khái niệm Responsive Design và Adaptive Design

1. Responsive Design (Thiết kế đáp ứng)

- **Theo Flutter Docs (Fit):**
Sắp xếp và định cỡ các phần tử giao diện sao cho **vừa vặn với không gian có sẵn** (*Available Screen Space / Constraints*).
- **Theo Web Design (Fluid – Ethan Marcotte):**
Bố cục duy nhất, co giãn **liên tục, mượt mà như chất lỏng**.
- **Công cụ Flutter cốt lõi:**
`Expanded`, `Flexible`, `Wrap`, `AspectRatio`, `FractionallySizedBox`.

2. Adaptive Design (Thiết kế thích ứng)

- **Theo Flutter Docs (Usable):**
Lựa chọn bố cục và thành phần điều khiển phù hợp để **tối ưu khả năng sử dụng** theo ngữ cảnh thiết bị (*Platform, Input, Screen*).
- **Theo cộng đồng Web (Discrete):**
Mỗi ngưỡng một bố cục riêng, **nhảy bậc rời rạc** tại các ngưỡng Breakpoint; rẽ nhánh hệ điều hành.
- **Cơ chế thực thi trong Flutter:**
Rẽ nhánh `if/else` theo Breakpoint, theo Nền tảng (`defaultTargetPlatform`, `kIsWeb`), phím tắt.

Bản chất công cụ đo lường: `MediaQuery` và `LayoutBuilder` là các **công cụ đo đạc không gian (Measure)**; giao diện là *Responsive* hay *Adaptive* phụ thuộc vào chiến lược cấu trúc **sau khi đo** (co giãn liên tục hay rẽ nhánh nhảy bậc)!

Bảng đối chiếu chuyên sâu: Responsive vs. Adaptive

Tiêu chí	Responsive UI (Đáp ứng)	Adaptive UI (Thích ứng)
1. Bản chất thay đổi	Thay đổi **kích thước** phần tử trên **cùng một cấu trúc** cây Widget duy nhất.	Thay đổi **cấu trúc** cây Widget (hoán đổi hoặc thay thế Widget này bằng Widget khác).
2. Tính chất chuyển dịch	Mượt mà, liên tục theo từng đơn vị dp khi người dùng kéo dãn cửa sổ (*Fluid layout*).	Nhảy bậc, rời rạc khi kích thước vượt qua các ngưỡng điểm ngắt định trước (*Breakpoints*).
3. Minh chứng trong dự án	Độ rộng mỗi thẻ học phần trong `GridView` tự co dãn để lấp đầy cột khả dụng.	Hoán đổi `NavigationBar` ↔ `NavigationRail`; hoán đổi `ListView` ↔ `GridView`.
4. Mối quan hệ thực tế	**Phân định theo Phạm vi đo lường (Scope)**: Cấp độ Vĩ mô (*Macro* — toàn cửa sổ) và Cấp độ Vi mô (*Micro* — vùng nội dung cục bộ). Ở cả 2 cấp độ đều kết hợp linh hoạt cả Responsive lẫn Adaptive!	

Cơ chế hoạt động của MediaQuery

- **Bản chất kỹ thuật:**
  - `MediaQuery` là một **InheritedWidget** ở gốc ứng dụng, cung cấp dữ liệu về toàn bộ cửa sổ (*Window Viewport*).
  - **Đơn vị đo lường:** Đo bằng **Logical Pixels (dp)**, không phải pixel vật lý. Ví dụ: Màn hình Windows scale 125% thì 1920px vật lý tương ứng 1536dp.
- **Các thuộc tính cốt lõi & API liên quan:**
  - `size`: Kích thước tổng thể cửa sổ (`width`, `height`).
  - `orientation`: Hướng toàn cửa sổ (`MediaQuery.orientationOf`) $\longleftrightarrow$ Phân biệt với hướng cục bộ theo Widget cha (`OrientationBuilder`).
  - `padding` & `viewInsets`: Vùng khuyết phần cứng (`SafeArea`) và vùng bàn phím ảo.
  - `textScaler`: Tỉ lệ cỡ chữ theo trợ năng (*Accessibility*).

Tối ưu hiệu năng: MediaQuery.sizeOf

1. Cách tiếp cận cũ ($<$ Flutter 3.10):

```
MediaQuery.of(context).size
```

$\rightarrow$ Đăng ký lắng nghe **toàn bộ** `MediaQueryData`. Bất kỳ thuộc tính nào đổi (bàn phím ảo bật lên, padding đổi) $\rightarrow$ Rebuild thừa các widget phụ thuộc!

2. Chuẩn tối ưu mới (Flutter 3.10+):

```
MediaQuery.sizeOf(context)
```

$\rightarrow$ Ứng dụng cơ chế **InheritedModel**, chỉ rebuild khi kích thước thay đổi. **Loại bỏ rebuild do bàn phím ảo và padding!**

Cơ chế hoạt động của LayoutBuilder

Quy tắc vàng trong thuật toán Layout của Flutter:

"Constraints go down. Sizes go up. Parent sets position."

(Ràng buộc truyền từ cha xuống con $\longrightarrow$ Con tự tính kích thước báo lên cha $\longrightarrow$ Cha xác định vị trí cho con)

- **Bản chất kỹ thuật của LayoutBuilder:**
  - Hoãn (*defers*) việc xây dựng cây Widget con cho đến **pha Layout** (*Layout phase*).
  - Cung cấp trực tiếp đối tượng `BoxConstraints` mà **Widget cha truyền xuống** (`minWidth`, `maxWidth`, `minHeight`, `maxHeight`).
- **Ý nghĩa kiến trúc thành phần:**
  - Giúp xây dựng các **Component độc lập** (***Decoupled Widgets***).
  - Component tự quyết định bố cục dựa trên **không gian khả dụng thực tế của chính nó**, không bị phụ thuộc vào kích thước toàn cửa sổ.



Quy trình 3 bước chuẩn Flutter: Abstract → Measure → Branch

Triết lý kiến trúc chính thức của Google Flutter trong thiết kế giao diện thích ứng:

1. ABSTRACT

(Trừu tượng hóa)

- **Mục tiêu:**

Xác định các Widget cần thích ứng và tách dữ liệu dùng chung để tái sử dụng độc lập ở nhiều ngữ cảnh mà không phụ thuộc cứng vào màn hình cha.

- **Ánh xạ mã nguồn:**

→ Tách Widget: `Card`, `Content`
→ Tách Dữ liệu: `_navItems` dùng chung

2. MEASURE

(Đo đạc không gian)

- **Mục tiêu:**

Đo đạc chính xác diện tích khả dụng tại đúng phạm vi phụ trách (*Macro* hoặc *Micro*).

- **Ánh xạ mã nguồn:**

→ **Macro (Toàn cửa sổ):**
`MediaQuery.sizeOf(context)`
→ **Micro (Vùng nội dung):**
`LayoutBuilder (maxWidth)`

3. BRANCH

(Rẽ nhánh quyết định)

- **Mục tiêu:**

Dựa trên kết quả đo, đưa ra quyết định: Hoán đổi cấu trúc widget hoặc co giãn thông số.

- **Ánh xạ mã nguồn:**

→ `isWide ? NavRail : NavBar`
→ `w < 500 ? List : Grid`
→ `cols = (w/250).clamp(2,4)`

Tổng kết quy trình: Ứng dụng hiện thực hóa đồng bộ 3 bước Abstract, Measure, Branch ở cả cấp độ toàn cửa sổ (Scaffold) lẫn vùng chứa cục bộ (Content).

So sánh Phạm vi đo lường & Hệ thống Breakpoint (MD3)

Tiêu chí	MediaQuery (Toàn cục - Macro)	LayoutBuilder (Cục bộ - Micro)
1. Phạm vi & Thời điểm	Toàn bộ cửa sổ ứng dụng (*Viewport*) • Đánh giá ở pha Build qua `InheritedModel`.	Vùng chứa do cha cấp phát (*BoxConstraints*) • Hoãn đánh giá đến pha Layout.
2. Tình huống 840dp (Demo thực tế)	Vẫn thấy toàn màn hình ≥ 840dp $\rightarrow$ **Dễ tính thừa cột, gây vỡ/tràn viền!**	Đo đúng vùng nội dung còn ≈ 583dp (đã trừ Rail 256dp) $\rightarrow$ **Tự hạ 2 cột, bảo toàn bố cục!**

1. Khái niệm Breakpoint & Mốc Nội dung

• **Định nghĩa Breakpoint:**
Là các **ngưỡng kích thước định lượng (dp)** để quyết định rẽ nhánh cấu trúc hoặc co giãn giao diện.

• **Nguyên tắc cốt lõi:**
Tư duy theo **không gian khả dụng**, tuyệt đối tránh phỏng đoán phần cứng (`isTablet/isPhone`).

• **Mốc nội dung nhóm chọn (Content-driven):**
500dp (ngưỡng chia List/Grid) & **250dp** (độ rộng cột tối thiểu, chưa trừ padding/khoảng cách).

2. Chuẩn Window Size Classes (Material 3)

• **Phân loại Chiều rộng cửa sổ (Window Width):**
– **Compact**: < 600dp • **Medium**: $600 - 839$dp
– **Expanded**: $840 - 1199$dp (bắt đầu ≥ 2 panes)
– **Large**: $1200 - 1599$dp • **Extra-large**: ≥ 1600dp

• **Chiều cao (Height):** Compact (< 480), Medium ($480 - 899$), Expanded (≥ 900dp).

• **Áp dụng:** Demo dùng mốc 600 & 840dp của M3 cho khung ngoài (Scaffold/Rail).

Các mẫu bố cục thích ứng & Canonical Layouts (M3)

1. Ba mẫu Canonical Layouts chuẩn M3

- **Supporting Pane:**
Khu vực nội dung chính đi kèm một ngăn thông tin phụ hỗ trợ thao tác bên cạnh.
- **List-Detail:**
Danh sách chọn bên trái, khung chi tiết bên phải trên cùng một màn hình rộng (≥ 2 panes).
- **Feed / Grid (Dự án demo):**
Bố cục dạng thẻ lưới hiển thị luồng thông tin, tự co dãn số cột theo không gian khả dụng.

2. Phân bổ Pane & Công thái học hiển thị

- **Khuyến nghị số Pane theo Size Class:**
Compact & Medium: 1 pane duy nhất.
Expanded & Large: 2 panes song song.
Extra-large (≥ 1600dp): 2 đến 3 panes.
- **Giới hạn chiều rộng đọc (Readability):**
Tránh để dòng văn bản trải dài hết màn hình rộng; nên giới hạn độ rộng đọc tối ưu (thường khoảng $600 - 800$dp theo quy tắc kinh nghiệm; tab Thiết bị demo giới hạn 580dp) bằng `ConstrainedBox`.

Nguyên tắc thiết kế theo Phương thức nhập liệu (Input Modalities)

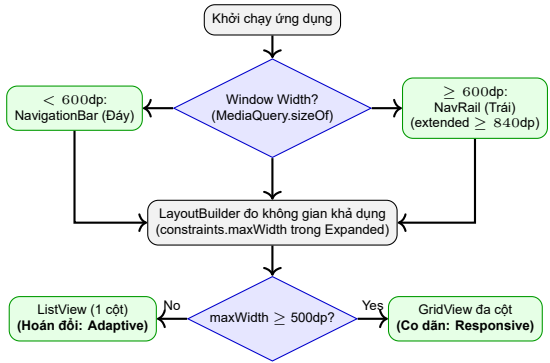
Tiêu chí	Cảm ứng (Touch - Mobile/Tablet)	Chuột & Bàn phím (Pointer/Keyboard - Desktop)
1. Tiết diện & Vùng tương tác	Đầu ngón tay có tiết diện lớn → **Bắt buộc vùng chạm tối thiểu** $\geq 48 \times 48$**dp** (`kMinInteractiveDimension`).	Con trỏ chuột chính xác từng pixel → Khi dùng `adaptivePlatformDensity`, Flutter tự chuyển `VisualDensity` sang `compact` để tăng mật độ dữ liệu.
2. Trạng thái tương tác	**Không có trạng thái Hover (rê chuột)**, nhưng hỗ trợ đa dạng cử chỉ: tap, double-tap, long-press, drag/swipe và pinch zoom.	Bổ sung **Hover** (`MouseRegion`), đổi con trỏ chuột (`SystemMouseCursors.click`), `Tooltip`, chuột phải (`onSecondaryTap`) và cuộn `Scrollbar`.
3. Điều hướng & Phím tắt	Thao tác vuốt cuộn màn hình (*Fling / Pan gesture*).	Di chuyển tiêu điểm qua phím `Tab` (`FocusTraversalGroup`) và kích hoạt lệnh tức thì qua **Phím tắt** (`CallbackShortcuts`).
4. Thiết bị lai (Hybrid Devices)	**Laptop Windows cảm ứng (Chuột + Chạm)**: Sàn tối thiểu cho mọi thành phần chạm được bắt buộc $\geq 48 \times 48$dp (`kMinInteractiveDimension`). Mật độ `compact` chỉ áp dụng cho hiển thị dữ liệu chuột, tuyệt đối không được phá vỡ ngưỡng chạm công thái học này.	

Kiến trúc ứng dụng Thực hành & Luồng chọn bố cục

Cấu trúc 4 tệp mã nguồn dự án:

- `breakpoints.dart`: Quản lý mốc M3 (`compact = 600`, `medium = 840`) & mốc nội dung nhóm chọn (`contentSplit = 500`, `minCardWidth = 250`).
- `adaptive_home_screen.dart`: Điều phối Macro 3 tầng (`MediaQuery`) & Bắt phím tắt `Alt+1`, `Alt+2`.
- `responsive_content.dart`: Điều phối Micro (`LayoutBuilder`) & Tính số cột tự động.
- `interactive_item_card.dart`: Thẻ tương tác chuột (`MouseRegion`), chạm chuẩn công thái học (`kMinInteractiveDimension`).

Sơ đồ luồng quyết định 3 tầng (*Decision Flowchart*)



Phân tích Mã nguồn Trọng tâm

1. Thích ứng 3 cấp độ với MediaQuery.sizeOf (Adaptive Vĩ mô):

```
final Size size = MediaQuery.sizeOf(context);
// M3 Window Size Classes: 600dp (Medium), 840dp (Expanded)
final isWide = size.width >= AppBreakpoints.compactBreakpoint;
final isExpanded = size.width >= AppBreakpoints.mediumBreakpoint;

bottomNavigationBar: isWide ? null : NavigationBar(
  destinations: [for (final i in _navItems)
    NavigationDestination(...)],
),
body: Row(children: [
  if (isWide) NavigationRail(
    extended: isExpanded, // Mo rong (~256dp)
    labelType: isExpanded ? NavigationRailLabelType.none
                : NavigationRailLabelType.all,
    destinations: [for (final i in _navItems)
      NavigationRailDestination(...)],
  ),
  Expanded(child: _selectedIndex == 0
      ? ResponsiveContent() : _buildDeviceSpecs()),
]),
```

2. Bố cục Micro bằng LayoutBuilder (Adaptive + Responsive):

```
LayoutBuilder(builder: (context, constraints) {
  final double w = constraints.maxWidth;
  // Adaptive vi mo: Hoan doi ListView khi maxWidth < 500dp
  if (w < AppBreakpoints.contentSplit) return
        _buildSingleColumnList();
  // Responsive vi mo: Tinh cot theo minCardWidth (250dp)
  return _buildMultiColumnGrid(w);
});
```

3. Bắt phím tắt Windows (Keyboard Input):

```
CallbackShortcuts(
  bindings: {
    // Alt+1: Chuyen Tab Bo cuc; Alt+2: Chuyen Tab Thiet bi
    const SingleActivator(LogicalKeyboardKey.digit1, alt: true):
      () => setState(() => _selectedIndex = 0),
    const SingleActivator(LogicalKeyboardKey.digit2, alt: true):
      () => setState(() => _selectedIndex = 1),
  },
  child: Focus(autofocus: true, child: Scaffold(...)),
)
```

4. Chuột & Chốt công thái học (kMinInteractiveDimension):

```
MouseRegion(
  cursor: SystemMouseCursors.click,
  onEnter: (_) => setState(() => _isHovered = true),
  onExit: (_) => setState(() => _isHovered = false),
  child: ConstrainedBox(
    // Bao ve vung tuong tac toi thieu >= 48dp chuan Material
    constraints: const BoxConstraints(
      minHeight: kMinInteractiveDimension,
      minWidth: kMinInteractiveDimension,
    ),
    child: InkWell(onTap: () {}, child: ...),
  ),
)
```

Kịch bản Thực nghiệm & Live Demo Trực tiếp

Chiều rộng cửa sổ	Vùng nội dung (Micro)	Lưới thẻ	Hiện tượng quan sát & Bản chất kỹ thuật
< 500dp	Bằng cửa sổ (< 500dp)	**1 cột**	`NavigationBar` ở đáy; `LayoutBuilder` chọn **`ListView` 1 cột** (Adaptive vi mô).
$500 - 599$dp	Bằng cửa sổ ($500 - 599$dp)	**2 cột**	`NavigationBar` vẫn ở đáy, nhưng nội dung đã nhảy sang **`GridView` 2 cột**!
$600 - 839$dp	$\approx 519 - 758$dp (trừ Rail ≈ 80, Divider 1)	**2 → 3 cột**	Chuyển sang `NavigationRail` thu gọn bên trái. Tại $831 - 839$dp vùng nội dung ≥ 750dp → **Tăng lên 3 cột**.
840dp (Mở rộng)	**≈ 583dp (trừ Rail ≈256, Divider 1)**	**Tụt về 2 cột!**	**Hiện tượng thực nghiệm quan trọng**: Cửa sổ to ra nhưng lưới bị thu hẹp! Chứng minh: `MediaQuery` thấy rộng hơn nhưng `LayoutBuilder` thấy hẹp lại do NavigationRail bung nhãn chữ chiếm chỗ.
≈ 1007dp	≥ 750dp	**3 cột**	Không gian khả dụng vượt lại mốc 750dp, lưới tự động tăng lại lên 3 cột (Responsive vi mô).
≥ 1257dp	≥ 1000dp	**4 cột**	Vùng nội dung vượt 1000dp → Đạt tối đa 4 cột theo hàm `clamp(2, 4)`.

Thử nghiệm phím tắt: Bấm **Alt+2** chuyển tức thì sang tab kiểm tra thông số thiết bị; bấm **Alt+1** quay lại tab bố cục.

Tổng kết Đề tài & Hạn chế Nghiên cứu

1. Ba thông điệp then chốt

- **Quy trình 3 bước chuẩn:**
Abstract → **Measure** → **Branch**. Đồng bộ cả cấp độ toàn cửa sổ (*Scaffold*) lẫn vùng nội dung cục bộ (*Content*).

- **Đúng công cụ đo đạc:**
Dùng `sizeOf` cho toàn cửa sổ (tránh rebuild thừa) và `LayoutBuilder` đo không gian khả dụng thực tế.

- **Đa dạng phương thức nhập liệu:**
Bảo vệ vùng chạm $\geq 48 \times 48$dp (`kMinInteractiveDimension`); tối ưu chuột (`compact`, hover) và phím tắt Desktop.

2. Hạn chế đề tài & Hướng mở rộng

- **Mốc ngắt nội dung:**
500dp và 250dp là mốc cố định theo thẻ demo; chưa đo kích thước nội dung thực tế (*intrinsic size*) theo độ dài văn bản.

- **Môi trường thực nghiệm:**
Tập trung phát triển và kiểm thử trên **Windows Desktop**; chưa kiểm thử trên màn hình cảm ứng vật lý thực tế.

- **Ràng buộc lưới & Trợ năng:**
`GridView` áp ràng buộc chặt (*tight*) từ `childAspectRatio` nên `minHeight: 48` chỉ mang tính phòng vệ; chưa thử `textScaler`.

Tài liệu tham khảo (References)

1. **Google Flutter Official Documentation:**
   "Adaptive and responsive design in Flutter" – Hướng dẫn tổng quan về thiết kế giao diện đa nền tảng trong Flutter.
   `https://docs.flutter.dev/ui/adaptive-responsive`
2. **Google Flutter Official Documentation:**
   "General approach to adaptive apps" – Quy trình thiết kế 3 bước chuẩn: Abstract → Measure → Branch.
   `https://docs.flutter.dev/ui/adaptive-responsive/general`
3. **Google Material Design 3 & Android Developers Specifications:**
   "Window size classes and canonical layouts" – Chuẩn phân loại kích thước cửa sổ theo chiều rộng và chiều cao (480/900dp).
   `https://m3.material.io/foundations/layout/applying-layout/window-size-classes`
   `https://developer.android.com/develop/adaptive-apps/guides/use-window-size-classes`
4. **Ethan Marcotte (2011) & Aaron Gustafson (2011):**
   "Responsive Web Design" (Lưới co dãn – Fluid Grid) & *"Adaptive Web Design"* (Tăng cường lũy tiến – Progressive Enhancement theo thiết bị); kết hợp khái niệm bố cục thích ứng rời rạc (Discrete Layouts) phổ biến trong thiết kế web.
5. **Flutter Framework API Reference:**
   Tài liệu kỹ thuật các lớp `MediaQuery`, `LayoutBuilder`, `VisualDensity`, `NavigationRail`, `CallbackShortcuts`.
   `https://api.flutter.dev`
6. **Kho lưu trữ mã nguồn Đề tài (GitHub):**
   `https://github.com/KhoaLe246205/cross-platform/tree/main/responsive_adaptive`

XIN CHÂN THÀNH CẢM ƠN!

QUÝ THẦY VÀ CÁC BẠN ĐÃ LẮNG NGHE